A PRACTICAL PUBLISHING GUIDE

# How to Make a Static Site in Replit for Vercel and Add a Domain

Build your site in Replit, send it to GitHub, publish it on Vercel, and connect a domain you own.

Beginner-friendly · Updated October 3, 2026

CONTENTS


1. Start here: a simple route to a live site
2. Chapter 1: Create the site in Replit
3. Chapter 2: Test the site and production build
4. Chapter 3: Connect Replit to GitHub
5. Chapter 4: Import the repository into Vercel
6. Chapter 5: Connect a custom domain
7. Chapter 6: Troubleshooting and launch checklist
8. References and key terms

# Start here: a simple route to a live site

This guide is for a website made from files delivered to a visitor's browser: HTML, CSS, JavaScript, images, and fonts. It covers both plain HTML and a Vite-based project.

The route is:

1. **Replit** — write and preview the site.
2. **GitHub** — save and share the source code.
3. **Vercel** — build and host the site.
4. **Your domain** — point a memorable address to the site.

## What “static” means

A static site has no server-side code that must run for each visit. A contact form, login, database, or private API key may need a separate backend. Do not put secrets in browser code.

# Chapter 1: Create the site in Replit

Start with a Replit project that matches the kind of site you need. Plain HTML is the simplest option; Vite is a common choice for React, Vue, and other front-end projects.

## Choose a project type

- **Plain HTML:** usually includes `index.html`, CSS, JavaScript, and assets. Vercel serves the folder containing `index.html`.
- **Vite or React:** usually includes `package.json`, `src/`, and `index.html`. The production build commonly goes into `dist/`.

## Build a first version

1. Create a new project or open the existing site in Replit.
2. Make the main page useful on mobile and desktop. Use semantic headings, readable contrast, and descriptive text for images.
3. Keep the project focused on front-end files if you intend to publish it as a static site.
4. Open Replit's preview and click through the whole site—not just the home page.

## Optional Replit Agent prompt

> Build a responsive static website for [purpose]. Use [plain HTML/CSS/JavaScript or Vite/React]. Do not add a database, login, or server routes. For Vite, make sure the production build creates a `dist/` folder.

## Keep the layout organized

```
my-site/
├── index.html
├── package.json       # for projects with a build tool
├── src/               # source code for Vite apps
├── public/            # static assets for Vite
└── styles.css         # common in plain HTML sites
```

Existing projects can use a different layout. Know which folder contains the app's `package.json` and which folder contains the final `index.html`.

# Chapter 2: Test the site and production build

Replit's preview is for development. Before connecting Vercel, make sure the project can produce—or already contains—the exact files a web host should serve.

## Test the visible site

- Check the home page at phone and desktop widths.
- Test navigation, links, buttons, images, and forms.
- Look for missing fonts, broken images, and errors in the browser console.
- Try every page directly by its URL if the site has multiple routes.

## Build a Vite site in Replit Shell

Use the package manager that matches the lockfile already in the project:

```
# npm project
npm install
npm run build

# pnpm project (use this instead when the project uses pnpm)
pnpm install --frozen-lockfile
pnpm run build

# Confirm the output exists
ls dist
```

A successful Vite build normally puts the ready-to-publish files in `dist/`. If the build fails, fix that first; Vercel runs a similar build when it deploys.

## Do not mix package managers casually

If the project has `pnpm-lock.yaml`, use pnpm; if it has `package-lock.json`, use npm. Mixing them can create conflicting lockfiles and different builds.

## Know the static-site boundary

HTML and browser JavaScript are public to every visitor. If the site needs protected credentials or server-only work, keep that part in a backend service and call it securely. A static export alone cannot run your Replit server code.

# Chapter 3: Connect Replit to GitHub

Vercel can watch a GitHub repository and create a deployment when code is pushed. Replit's Git pane can initialize or connect a repository, review changes, commit them, and push them to the remote.

## Push your site

1. Open the Git tool in Replit. Current Replit documentation describes finding it from Tools and adding Git if it is not already open.
2. Initialize a repository or connect the Replit project to a GitHub repository.
3. Review the changed-file list. Stage the files you want to keep.
4. Write a short commit message, commit the changes, and push to GitHub.
5. Open the repository on GitHub and confirm that the latest files are there.

## Check before the first push

- Keep dependency lockfiles such as `pnpm-lock.yaml` or `package-lock.json` in the repository.
- Do not commit `.env` files, passwords, API keys, private certificates, or personal data.
- A public repository exposes its source. The deployed site's front-end files are public too, even if the repository is private.

After Vercel is connected, a push can trigger an automatic deployment. In a typical setup, pushes to the production branch update the live domain; other branches can be used for preview deployments.

Replit's Git pane and Git commands in Shell stay synchronized. Use one method you understand; do not push credentials by embedding tokens in repository URLs or source files.

# Chapter 4: Import the repository into Vercel

Create a Vercel project from the GitHub repository. Vercel detects many common front-end frameworks and can build the site on each push.

## Deploy from GitHub

1. Sign in to Vercel and start a new project from a Git repository.
2. Authorize GitHub access if asked, then select the repository you pushed from Replit.
3. Check the detected framework, the Root Directory, the build command, and the output folder before deploying.
4. Deploy, then open the Vercel-provided URL and test it.

## Check the build settings

- **Plain HTML:** choose the static/Other preset, use no build command unless the project has a build step, and select the folder containing `index.html`.
- **Vite:** the build command is usually `npm run build` or the equivalent for the project's package manager. The output folder is usually `dist`, unless the Vite config changes it.
- **Monorepos and subfolders:** set Vercel's Root Directory to the site folder—the one containing its project configuration. Confirm that the build can access any shared files it needs.

## Fix deep links in a single-page app

If a Vite single-page app works at `/` but returns 404 when you refresh a nested route, add this `vercel.json` at the project root. Use it only for a single-page app that should send routes to its client-side code.

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

## Keep private values out of the browser

Vite variables prefixed with `VITE_` are bundled for browser use. Never put a private API key in one. Add only values meant to be public in Vercel's project environment settings.

# Chapter 5: Connect a custom domain

You need a domain registered with Vercel or another domain registrar. Vercel will show the DNS records needed for your project. Follow the values on your own domain screen; do not rely on a copied value from an old tutorial.

## Add the domain

1. Open your Vercel project, then **Settings** → **Domains**.
2. Add the domain you own, such as `example.com`. Add `www.example.com` too if you want both versions.
3. Vercel displays the required DNS records and their status.
4. Sign in to the registrar or DNS provider where the domain's DNS is managed.
5. Add or update only the records Vercel requests. Save the changes.
6. Return to Vercel and wait for the domain to show as configured and its certificate to become ready.

## Apex and subdomain records

- The apex (root) address, such as `example.com`, typically uses an A record. Use the value Vercel displays.
- A subdomain, such as `www.example.com`, typically uses a CNAME record. Use the target Vercel displays.

## Protect email and existing services

Do not replace all domain nameservers just to point a website at Vercel. Changing nameservers can affect email and verification records. If you are changing nameservers intentionally, copy all existing records—including MX and TXT records—first.

## Choose one primary address

Decide whether visitors should use the root domain or `www`. Configure the other to redirect to it in Vercel so both addresses do not appear as separate versions of the same site.

DNS updates can take time to appear everywhere. Use the Vercel domain status as the source of truth, and verify the site over HTTPS after the records finish updating.

# Chapter 6: Troubleshooting and launch checklist

## Common issues

- **Vercel build fails:** open the build logs; check Root Directory, package manager, install command, and build script.
- **Blank page or broken assets:** check the browser console, output folder, file-name capitalization, and asset paths.
- **Nested route returns 404:** for a client-side SPA, use the Vercel rewrite in Chapter 4. Do not apply it blindly to a multi-page site.
- **Custom domain is invalid:** compare DNS records with Vercel's current instructions and remove only conflicting web records.
- **Latest edits are not live:** check that the commit was pushed to the connected repository and the production-branch deployment succeeded.

## Before you share the link

- The production URL loads on a phone and a desktop browser.
- Important links and navigation work; images and fonts load.
- Direct page visits and refreshes work for every route.
- The preferred domain and its alternate redirect correctly.
- HTTPS is active and the browser shows no certificate warning.
- No password, API key, or private data is included in the public site.
- A fresh GitHub push produces the expected Vercel deployment.

## Keep future updates simple

Edit in Replit → preview and test → commit and push to GitHub → check the Vercel deployment. If it is successful, Vercel serves the updated production site.

Replit also offers its own hosting options. This guide follows the separate GitHub-to-Vercel workflow; a static export does not include a backend that your application might require.

# References and key terms

## Official documentation

- Replit: Using the Git pane — connecting repositories, reviewing changes, committing, and pushing.
- Replit: Deployment types — how Replit describes static and server-based deployments.
- Vercel: Vite — build setup, environment variables, and Vite SPA rewrites.
- Vercel: Deploying GitHub projects — automatic deployments and production-branch behavior.
- Vercel: Adding and configuring a custom domain — project domain settings, DNS records, and domain verification.
- Amazon KDP: Supported eBook manuscript formats — supported manuscript files and Kindle Previewer guidance.

## Quick glossary

- **Static site:** files served to visitors without a per-visit application server.
- **Build:** the step that transforms source files into production-ready files.
- **Output directory:** the folder containing files the host should publish, often `dist/` for Vite.
- **Root Directory:** the folder inside a repository where Vercel should install and build the project.
- **DNS:** records that direct a domain name to the services hosting the site.
- **SPA:** a single-page application that handles page navigation in browser JavaScript.

When a Vercel dashboard or current official documentation shows different build or DNS values than this guide, follow the values shown for your project. Interface labels may change over time.

---

Prepared October 3, 2026. Interface labels may change over time.